COMP 3920 Database Systems 1

MySQL
Slow Inserts

Introduction:
The goal of this document is to serve as a walkthrough working with large INSERT statements, possibly as part of a restore of a large database, in MySQL 8.

We are going to insert data into MySQL with different methods to compare their load times.
There are 3 files to download from D2L and 3 ways we will try to import that data. Each file contains the definition for a database called **lotsofdata** and a table called **user** with 1 million rows.

Files to download:

1. Insert10.sql
2. Insert300.sql
3. Insert300nokeys.sql

Import data methods:

1. Uses **file redirect** from the command line and the **Insert10.sql** file.
2. Uses **file redirect** from the command line and the **Insert300.sql** file.
3. Uses **file redirect** from the command line and the **Insert300nokeys.sql** file.

Step 1: Lets first look at our Insert files.

If you open **Insert10.sql** you'll notice something like this:
INSERT INTO `user` (…) VALUES (1,…
INSERT INTO `user` (…) VALUES (11,…
INSERT INTO `user` (…) VALUES (21,…
INSERT INTO `user` (…) VALUES (31,…

For each INSERT statement, there are **10 sets of values**.

Compare that to the **Insert300.sql** and you'll notice that the Insert300.sql has instead **300 values per INSERT**.
INSERT INTO `user` (…) VALUES (1,…
INSERT INTO `user` (…) VALUES (301,…
INSERT INTO `user` (…) VALUES (601,…
INSERT INTO `user` (…) VALUES (901,…

Step 2: Before we start inserting, we need a way to measure how fast our imports are working and be able to estimate how long our inserts are going to take to import the 1,000,000 rows.

For each of our insert methods we will run our insert script, and then in an another window we can run the following query:
Timing Query #1:

```
SELECT @startTime := Now() AS StartTime,
@startCount := count(*) AS StartCount
FROM lotsofdata.user;
```

This will give us a starting time and the number or rows inserted at this point. We store these into variable `@startTime` and `@startCount`.

We will wait about 10 seconds or so and run this next query:
Timing Query #2:

```
SELECT @endTime := Now() AS EndTime,
@endCount := count(*) AS EndCount
FROM lotsofdata.user;
```

Timing Query #3:

```
SELECT @endTime AS EndTime, @endCount AS EndCount,
(timestampDiff(SECOND, @startTime, @endTime)) AS elapsedTime,
(@endCount - @startCount) AS newRows,
TIME_FORMAT(SEC_TO_TIME(1000000 / ((@endCount - @startCount) /
(timestampDiff(SECOND, @startTime, @endTime)))),'%Hh %im %ss') AS
EstimatedTime
FROM DUAL;
```

And this query will record an ending time, and the number of rows inserted at this point (stored as @endTime and @endCount). We now have enough information to estimate how long it will take to run our entire INSERT statement.

Time (s) = Rows / Speed - Where Speed is rows per second
Rows in this case is 1,000,000.

We calculate our speed by the number of rows inserted between running Timing Query #1 and Timing Query #2 divided by the elapsed time between running Timing Query #1 and Timing Query #2.

Number of rows Inserted = End Count – Start Count

```
newRows = @endCount - @startCount
```

Elapsed Time (s) = End Time - Start Time

```
elapsedTime = (timestampDiff(SECOND, @startTime, @endTime))
```

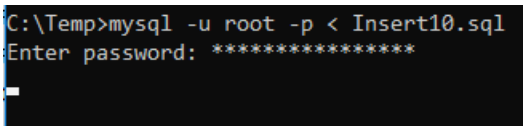
```
Estimated Time = 1000000 / ((@endCount - @startCount) /
(timestampDiff(SECOND, @startTime, @endTime)))
```

Step 3:

<u>INSERT Method 1</u>

Open a command line interface.
Run Insert10.sql using file redirect.

```
mysql -u root -p < Insert10.sql
```



How long will this query take to run?

Run Timing Query #1, wait at least 10 seconds, run Timing Queries #2 **<u>and</u>** #3.

Submit your screenshot (Timing Query #1):

01_Insert10_start.jpg

Submit

	StartTime	StartCount
▸	[illegible] 17:57:50	3860

Submit your screenshot (Timing Query #3):

02_Insert10_end.jpg

Submit

	EndTime	EndCount	elapsedTime	newRows	EstimatedTime
▸	[illegible] 17:58:52	[illegible]	[illegible]	[illegible]	[illegible]

You do **NOT** need to wait for this to finish.

Close the command line window. Using CTRL+C will probably not work; you'll need to close the window.

Step 4:
INSERT Method 2

Open a command line interface.
Run Insert300.sql using file redirect.

```
mysql -u root -p < Insert300.sql
```

```
C:\Temp>mysql -u root -p < Insert300.sql
Enter password: ****************
```

How long will this query take to run?

Run Timing Query #1, wait at least 10 seconds, run Timing Queries #2 **and** #3.

Submit your screenshot (Timing Query #1):

03_Insert300_start.jpg Submit

	StartTime	StartCount
▶	[illegible] 18:07:22	80400

Submit your screenshot (Timing Query #3):

04_Insert300_end.jpg Submit

	EndTime	EndCount	elapsedTime	newRows	EstimatedTime
▶	[illegible] 18:08:05	[illegible]	[illegible]	[illegible]	[illegible]

You do **NOT** need to wait for this to finish.

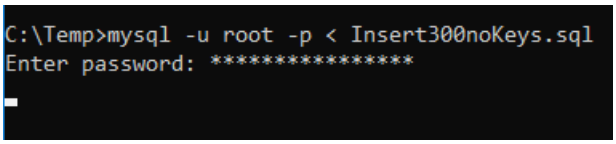
Close the command line window. Using CTRL+C will probably not work; you'll need to close the window.

Which Insert Method is faster: Method1 or Method2? Why is that?

Step 5:
<u>INSERT Method 3</u>

Open a command line interface.
Run Insert300nokeys.sql using file redirect.

```
mysql -u root -p < Insert300noKeys.sql
```

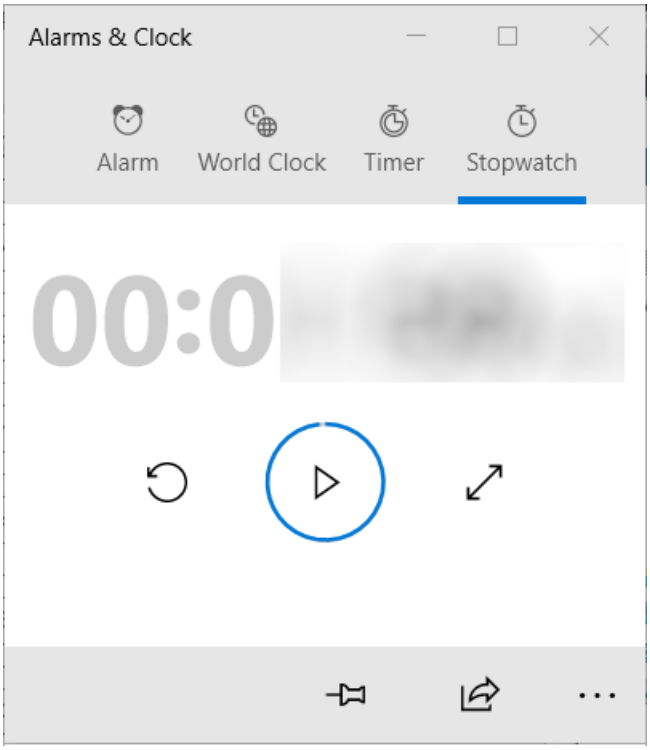


Let this Insert file finish.

This time, use a stopwatch app in windows or a stopwatch on your phone to time how long it takes to run this query. The Insert300noKeys.sql insert file is faster and may make it difficult to capture screenshots in the time. Also, the Timing Queries also take time to run and may not provide accurate information in this case.

When it's done, get a screenshot of your stop watch:

05_Insert300nokeys_stopwatch.jpg Submit



When it's done, get a screenshot of the number of rows in the `lotsofdata.user` table:

06_Insert300nokeys_complete.jpg

	count(*)
▶	

Submit

Which Insert Method is faster: Method2 or Method3? Why is that?

One of the disadvantages of disabling the keys and indexes on a table before the insert is that it is up the person doing the insert to ensure ALL data integrity. The table may now have inconsistent data, like duplicates in a unique field, invalid foreign keys – ex: children without parents or parents without their children, etc. Only disable keys if you know the data is correct BEFORE inserting.